자료구조

03 주차

리스트와 집합



3장. 리스트와 집합



3.1 리스트란?

3.2 파이썬의 리스트

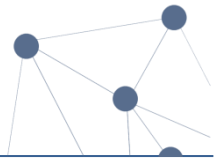
3.3 배열로 구현한 리스트

3.4 리스트의 응용: 라인 편집기

3.5 집합이란?

3.6 집합의 구현

3.1 리스트란?



- 가장 자유로운 선형 자료구조

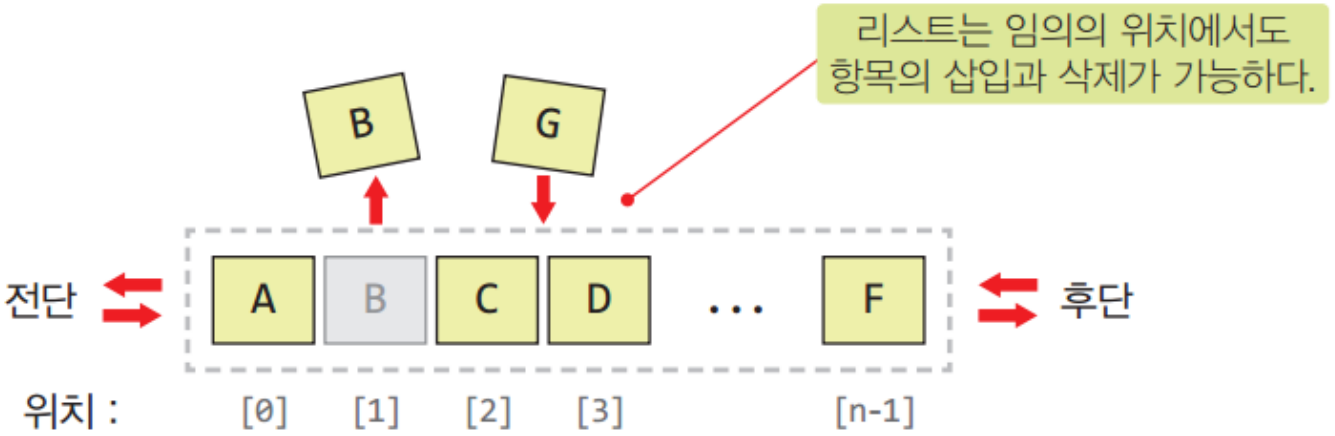
$$L = [item_0, item_1, item_2, \ldots, item_{n-1}]$$

- 리스트(list), 선형 리스트(linear list)
  - 순서를 가진 항목들의 모임
  - 항목들은 "위치"를 가짐
  - $L = [item_0, item_1, item_2, \ldots, item_{n-1}]$

- 집합은?
  - 항목간의 순서의 개념이 없음
  - 항목의 중복을 허용하지 않음
  - ➔ 선형 자료구조가 아님

리스트의 구조

- 리스트
  - 항목들이 순서대로 나열되어 있고, 각 항목들은 위치를 갖는다.



- Stack, Queue, Deque과의 차이점
  - 자료의 접근 위치

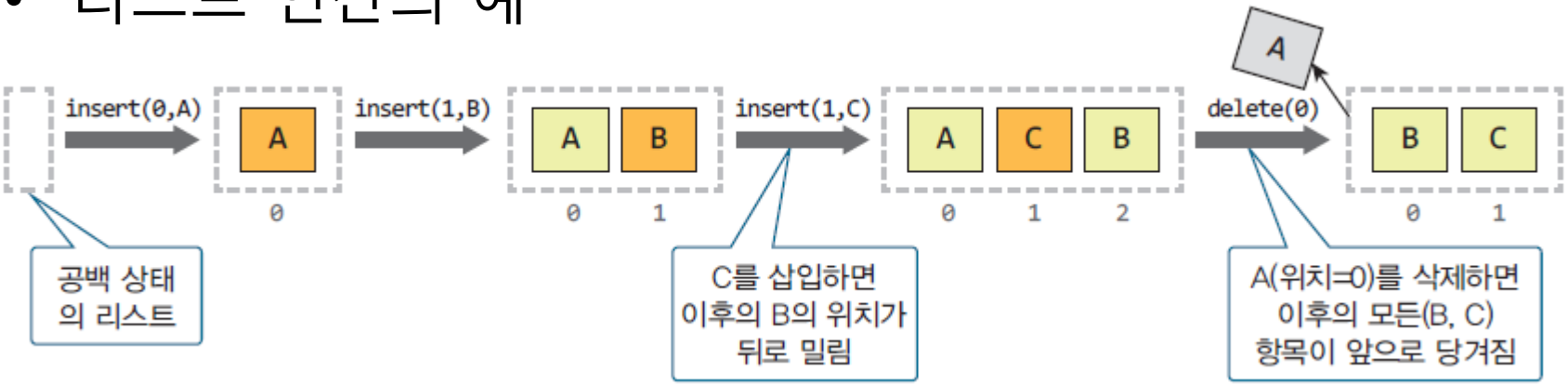
리스트의 추상 자료형



- 데이터
  - 같은 유형의 요소들의 순서 있는 모임
- 연산
  - insert(pos, e): pos 위치에 새로운 요소 e를 삽입한다.
  - delete(pos): pos 위치에 있는 요소를 꺼내고(삭제) 반환한다.
  - isEmpty(): 리스트가 비어 있는지를 검사한다.
  - isFull(): 리스트가 가득 차 있는지를 검사한다.
  - getEntry(pos): pos 위치에 있는 요소를 반환한다.

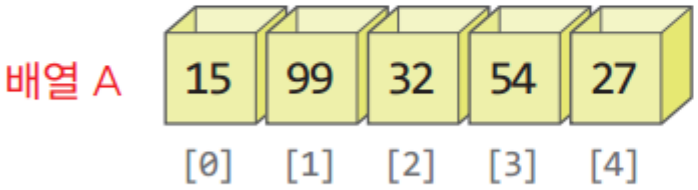
리스트의 연산

- 리스트의 연산
  - 삽입과 삭제: 리스트의 상태를 변경시킴
  - 연산의 위치를 지정해야 함
  - 위치(인덱스)는 보통 0부터 시작
  - 어떤 위치에 항목을 삽입/삭제하면 이후의 모든 자료들의 위치가 한 칸씩 밀리거나 당겨짐
- 리스트 연산의 예



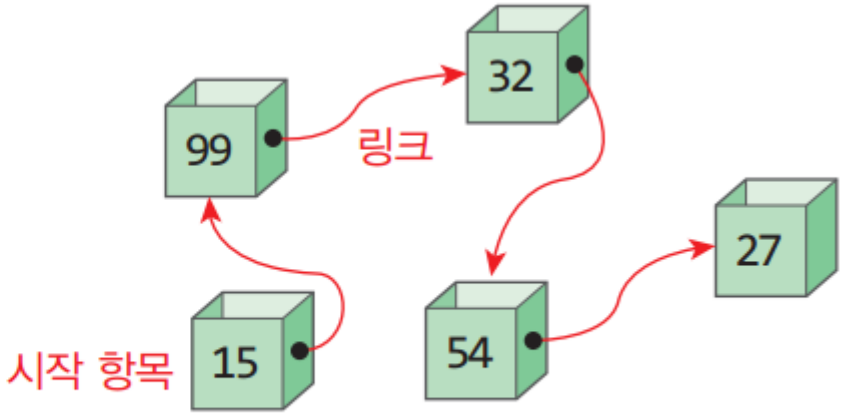
리스트 구현 방법

- 배열 구조
  - 구현이 간단
  - 항목 접근이 $O(1)$
  - 삽입, 삭제시 오버헤드
  - 항목의 개수 제한



배열 구조의 리스트

- 연결된 구조
  - 구현이 복잡
  - 항목 접근이 $O(n)$
  - 삽입, 삭제가 효율적
  - 크기가 제한되지 않음



연결된 구조의 리스트

파이썬의 연결리스트



- 파이썬에는 LinkedList라는 기본 자료형이 따로 없음
- 보통 **클래스**와 **노드(Node)**를 직접 만들어서 연결 리스트를 구현

배열 구조 리스트

```
[10][20][30][40][50]
 ↑
 시작 주소
```

```
data = [10, 20, 30, 40, 50]

print(data[3])   # 40
```

연결 구조 리스트

```
[10 | 다음] ──→ [20 | 다음] ──→ [30 | 다음] ──→ [40 | 다음]
```

```
10 → 20 → 30
          ↑
        찾았다!
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
┌──────────┬──────────┐
│   data   │   next   │
│    10    │    ●─────┼──→ 다음 노드
└──────────┴──────────┘
```

```
node1 = Node(10)
node2 = Node(20)
node3 = Node(30)

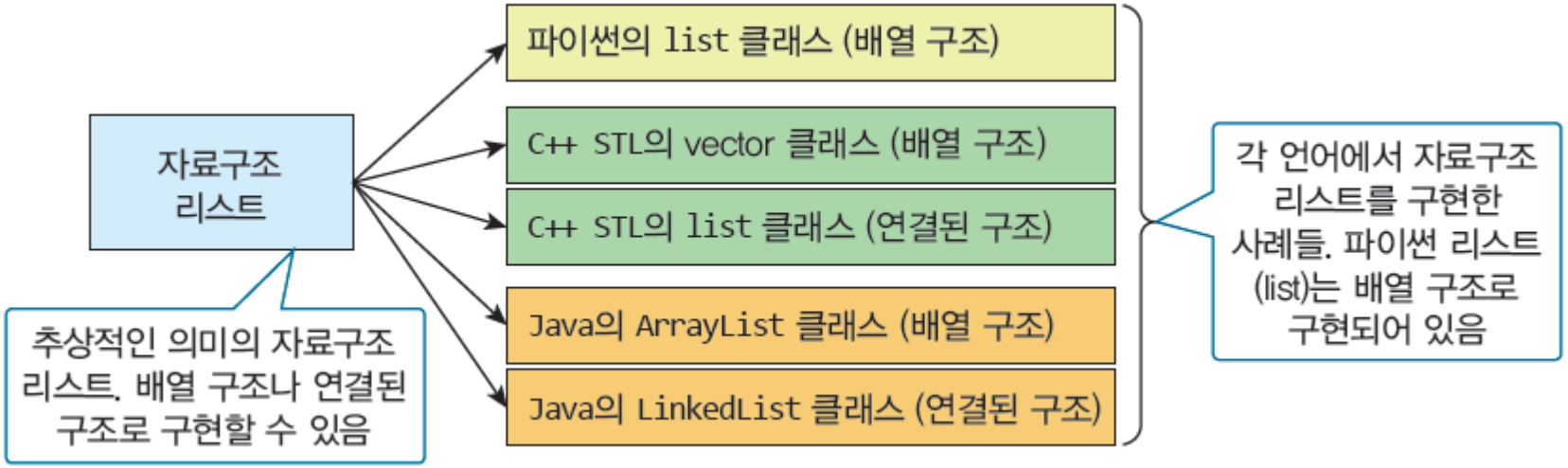
node1.next = node2
node2.next = node3
```

```
node1          node2          node3
┌──┬──┐      ┌──┬──┐      ┌──┬──┐
│10│ ●┼─────→│20│ ●┼─────→│30│None│
└──┴──┘      └──┴──┘      └──┴──┘
```

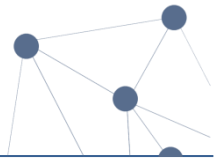
배열과 파이썬 리스트



- 파이썬의 리스트는 자료구조 리스트를 배열구조로 구현한 하나의 사례임.
- 자료구조 리스트를 구현한 다양한 사례들



3.2 파이썬 리스트



- 파이썬의 리스트는 스마트한 배열
- 파이썬 리스트 선언

```
A = [ 1, 2, 3, 4, 5 ]
B = [ 0 ] * 5
```

C 언어의 배열 선언

```
int A[5] = { 1, 2, 3, 4, 5 };
int B[5] = { 0, 0, 0, 0, 0 };
```

- 리스트의 크기 구하기
  - 내장 함수 len() 사용

```
print('파이썬 리스트 A의 크기는 ', len(A))
```

- 크기를 늘릴 수 있다

```
A.append(6)          # A = [1, 2, 3, 4, 5, 6]
A.append(7)          # A = [1, 2, 3, 4, 5, 6, 7]
A.insert(0, 0)       # A = [0, 1, 2, 3, 4, 5, 6, 7]
B.insert(9)          # B = [0, 0, 0, 0, 0, 9]
```

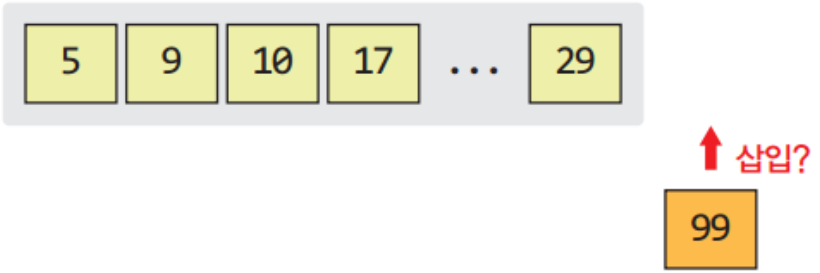
파이썬 리스트는 동적 배열



- 필요한 양보다 넉넉한 크기의 메모리를 사용!



(크기 < 용량) 인 상황에서의 항목 삽입

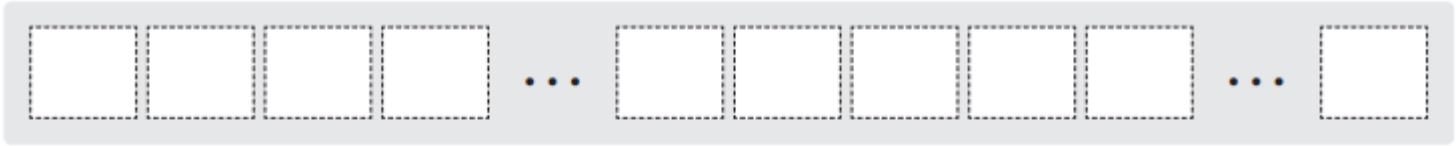


(크기 == 용량) 인 상황에서의 항목 삽입

- 남은 공간이 없으면 어떻게 삽입할까?

동적 배열 구조에서의 용량 증가 과정

Step1: 용량을 확장한 새로운 배열 할당. (예: 기존 배열 용량의 2배)



Step2: 기존의 배열을 새로운 배열에 복사



Step3: 항목을 삽입

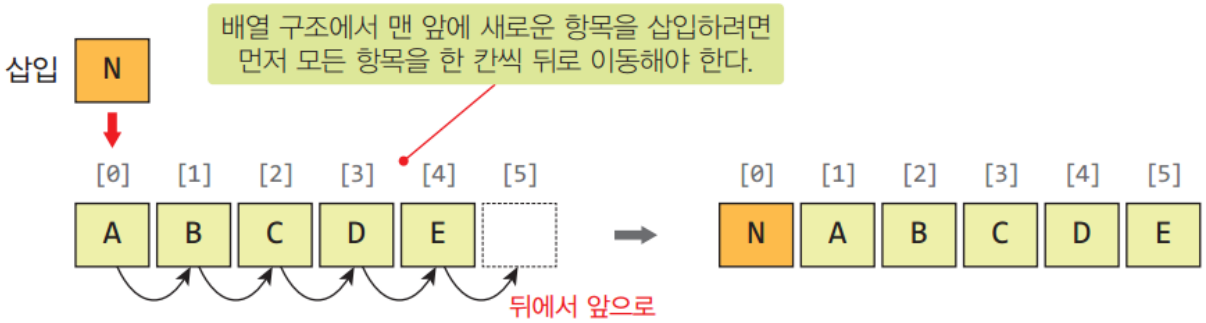


Step4: 기존 배열 해제, 리스트로 새 배열 사용

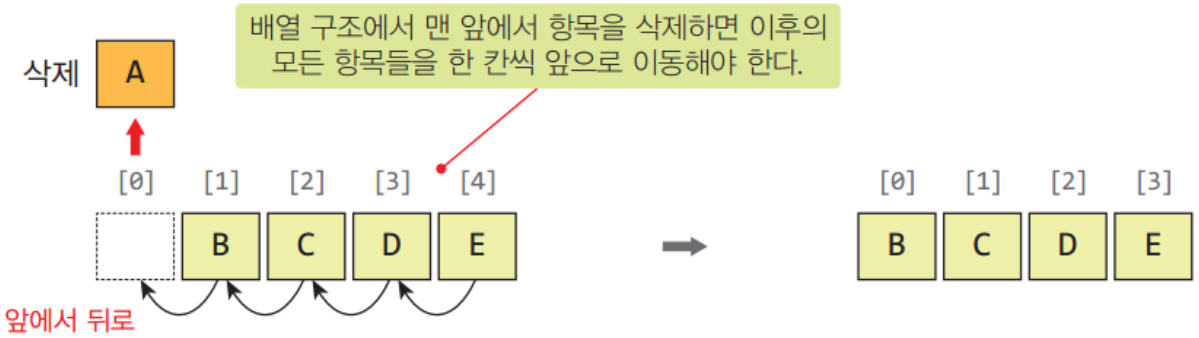
연산들의 시간 복잡도



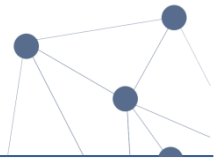
- append(e) : 대부분의 경우 $O(1)$
- insert(pos, e): $O(n)$



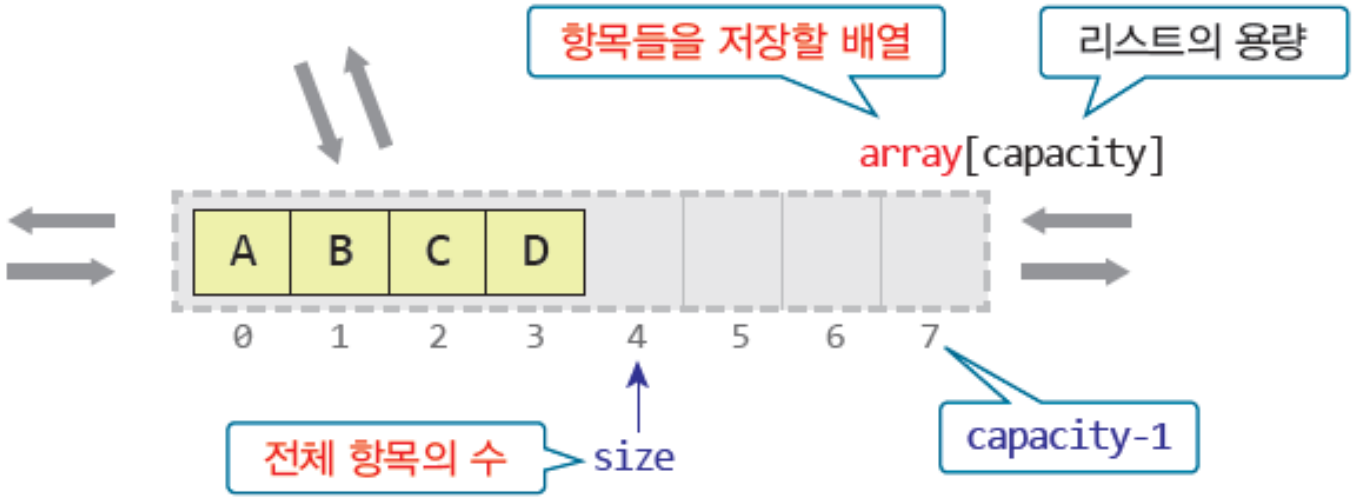
- pop(pos): $O(n)$



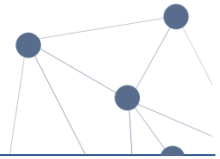
3.3 배열로 구현한 리스트



- 배열을 이용한 리스트의 구조
  - 용량이 고정된 리스트의 구조



배열로 구현한 리스트(함수 버전)

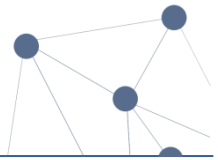


- 리스트 ADT를 배열 구조(파이썬 리스트 이용)로 구현

- 구현 방법: 전역 변수와 **함수** 이용 / **클래스** 이용
- 함수 버전
  - 리스트의 데이터를 전역 변수로 선언
  - 리스트의 연산은 일반 함수로 구현

- 클래스 버전
  - 리스트의 데이터를 클래스의 멤버 변수
  - 리스트의 연산은 클래스의 메소드(멤버 함수)

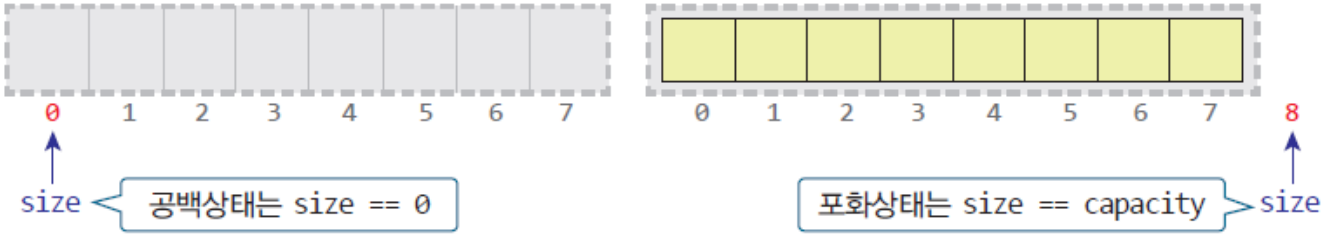
방법 1: 함수로 구현



- 리스트의 데이터: 전역 변수

```
capacity = 100
array = [None]*capacity
size = 0
```

- 포화상태와 공백 상태 검사




```
def isEmpty( ):
    if size == 0 : return True
    else : return False
```

```
def isFull( ):
    return size == capacity
```

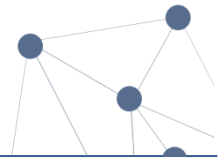
```python
# 리스트의 연산: 일반 함수
def isEmpty( ) :
    if size == 0 : return True  # 공백이면 True 반환
    else : return False         # 아니면 False 반환

def isFull( ) :
   return size == capacity      # 비교 연산 결과 반환

def getEntry(pos) :
    if 0 <= pos < size :
        return array[pos]
    else : return None

def insert( pos, e ) :
    global size                     # size는 전역변수
    if not isFull() and 0 <= pos <= size :
        for i in range(size, pos,-1) :
            array[i] = array[i-1]   # 한 칸씩 뒤로 옮김
        array[pos] = e              # pos위치에 새로운 요소 추가
        size += 1                   # 요소의 수가 하나 증가
    else :
        print("리스트 overflow 또는 유효하지 않은 삽입 위치")
        exit()

def delete( pos ) :
    global size                     # size는 전역변수
    if not isEmpty() and 0 <= pos < size :
        e = array[pos]
        for i in range(pos, size-1) :
            array[i] = array[i+1]   # 한 칸씩 앞으로 당김
        size -= 1                   # 요소의 수가 하나 감소
        return e
    else :
        print("리스트 underflow 또는 유효하지 않은 삭제 위치")
        exit()
```



교재 pp101
코드 3.1

테스트 코드

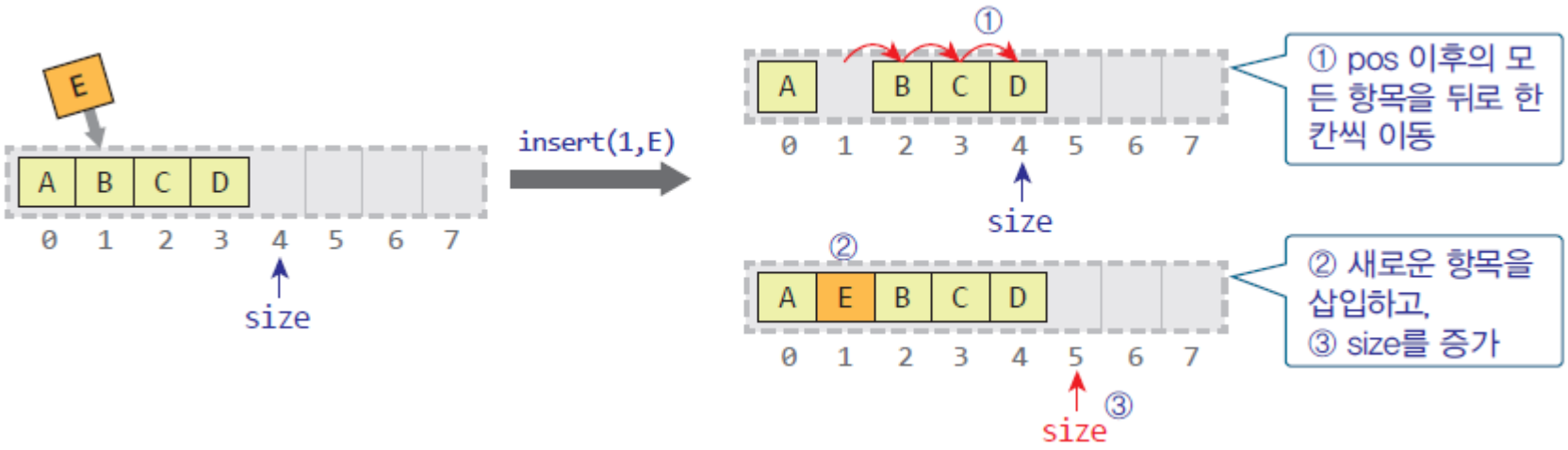


```python
# 테스트 코드
if __name__ == "__main__":
    print("최초   ", array[0:size])
    insert(0, 10)
    insert(0, 20)
    insert(1, 30)
    insert(size, 40)
    insert(2, 50)
    print("삽입x5 ", array[0:size])
    delete(2)
    print("삭제(2)", array[0:size])
    delete(size-1)
    print("삭제(E)", array[0:size])
    delete(0)
    print("삭제(0)", array[0:size])
```

[10]
[20, 10]
[20, 30, 10]
[20, 30, 10, 40]
[20, 30, 50, 10, 40]

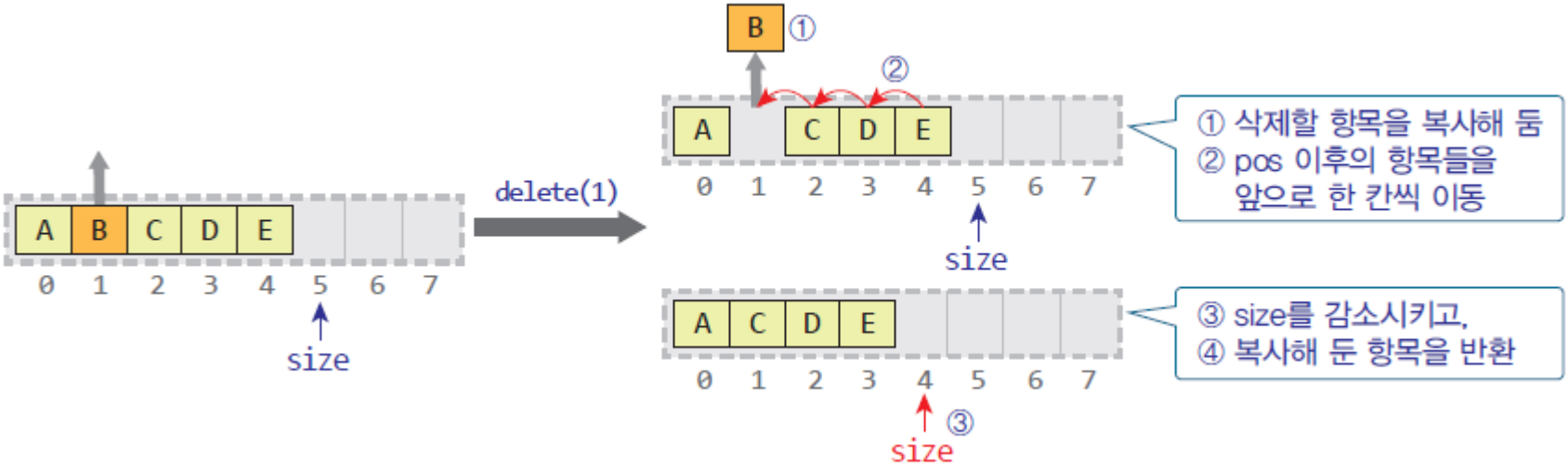
```
최초     []
삽입x5   [20, 30, 50, 10, 40]
삭제(2)  [20, 30, 10, 40]
삭제(E)  [20, 30, 10]
삭제(0)  [30, 10]
```

- 삽입 연산



  - 코드 3.1의 19~28행
  - O(n)

- 삭제 연산



  - 코드 3.1의 30~40행
  - O(n)

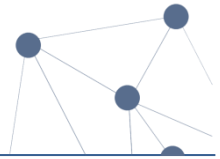
방법 2: 클래스로 구현



- 여러 개의 리스트를 사용하려면?
  - 전역 변수와 함수로는 어려움
  - 클래스가 자료구조를 구현하는 가장 좋은 방법

- 함수를 클래스로 변경하는 과정
  - 필요한 클래스를 선언한다.
  - 전역변수로 선언된 데이터 → 클래스의 멤버 변수(생성자에서)
  - 일반 함수로 구현된 연산 → 클래스의 멤버 함수
    - 첫 번째 매개변수로 **self** 추가
  - 멤버 함수에서 멤버 변수나 멤버 함수를 호출하기 위해 self사용

파일명: ArrayList.py



```python
class ArrayList:
    # 리스트의 데이터: 생성자에서 정의 및 초기화
    def __init__( self, capacity=100 ):
        self.capacity = capacity
        self.array = [None]*capacity
        self.size = 0

    # 리스트의 연산: 클래스의 메소드
    def isEmpty( self ):
       return self.size == 0

    def isFull( self ):
       return self.size == self.capacity

    def getEntry(self, pos) :
        if 0 <= pos < self.size :
            return self.array[pos]
        else : return None

    def replace(self, pos, e):
        if 0 <= pos < self.size:
            self.array[pos] = e
```

```python

    def insert( self, pos, e ) :
        if not self.isFull() and 0 <= pos <= self.size :
            for i in range(self.size, pos,-1) :
                self.array[i] = self.array[i-1]
            self.array[pos] = e
            self.size += 1
        else : pass

    def delete( self, pos ) :
        if not self.isEmpty() and 0 <= pos < self.size :
            e = self.array[pos]
            for i in range(pos, self.size-1) :
                self.array[i] = self.array[i+1]
            self.size -= 1
            return e
        else : pass

    #ArrayList 를 print 하려고 할때 자동 호출
    def __str__( self ) :
        return str(self.array[0:self.size])
```

__str__(self)
ArrayList 를 print 하려고 할때 자동 호출

테스트 프로그램(클래스 버전)



```
47  # 테스트 코드
48  if __name__ == "__main__":
49      L = ArrayList(50)
50
51      print("최초   ", L)
52      L.insert(0, 10)
53      L.insert(0, 20)
54      L.insert(1, 30)
55      L.insert(L.size, 40)
56      L.insert(2, 50)
57      print("삽입x5 ", L)
58      L.delete(2)
59      print("삭제(2)", L)
60      L.delete(L.size-1)
61      print("삭제(E)", L)
62      L.delete(0)
63      print("삭제(0)", L)
```

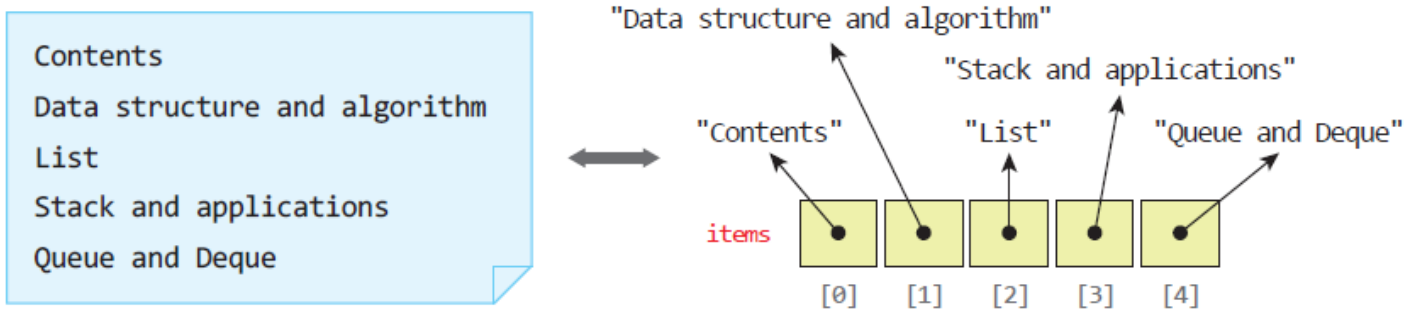
[10]
[20, 10]
[20, 30, 10]
[20, 30, 10, 40]
[20, 30, 50, 10, 40]

```
최초    []
삽입x5  [20, 30, 50, 10, 40]
삭제(2) [20, 30, 10, 40]
삭제(E) [20, 30, 10]
삭제(0) [30, 10]
```

3.4 리스트의 응용 : 라인 편집기



- 라인 단위로 입력이나 삭제를 할 수 있는 문서 편집기

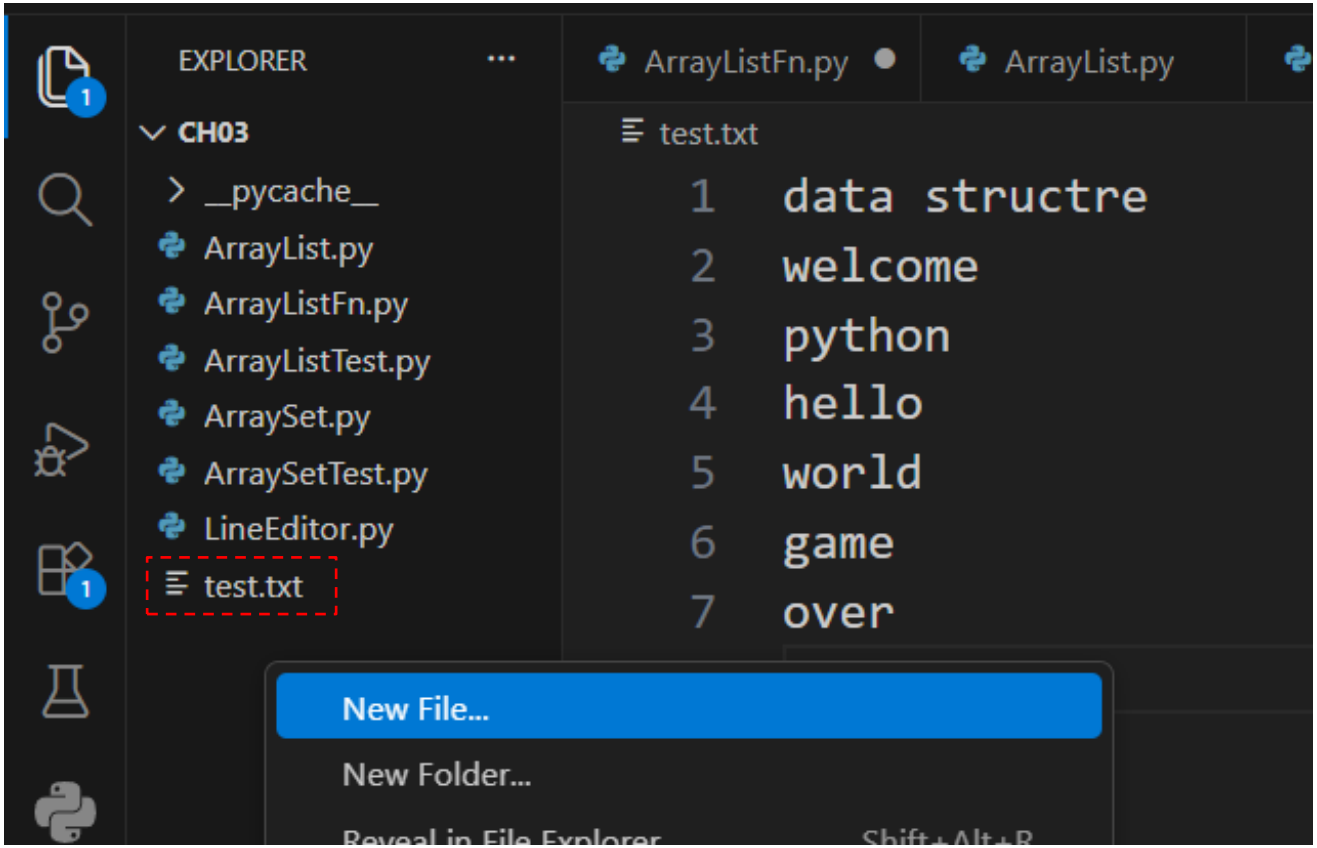


[그림 3.13] 리스트를 이용한 문서의 내부적인 표현

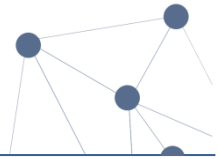
- 명령 i: 라인 삽입. 행 번호와 문자열을 입력하면 그 행에 문자열을 추가함
- 명령 d: 한 라인 삭제. 행 번호를 입력하면 그 행을 삭제
- 명령 r: 한 라인 변경. 행 번호와 문자열을 입력하면 그 행의 내용을 변경
- 명령 p: 현재 내용 출력. 현재 문서의 모든 내용을 라인 번호와 함께 출력
- 명령 l: 파일 입력. 지정된 (test.txt) 파일로부터 라인을 읽어 들임
- 명령 s: 파일 출력. 지정된 (test.txt) 파일로 편집 내용을 저장

샘플 데이터 파일 생성

파일명: test.txt



구현 코드



- ArrayList 클래스를 이용한 구현 예(pp110 코드 3.5)

```
from ArrayList import ArrayList

# 배열구조의 리스트를 이용한 라인 편집기 프로그램
list = ArrayList(1000)
while True :
    command = input("[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> ")

    if command == 'i' :
        pos = int( input("  입력행 번호: ") )
        str = input("  입력행 내용: ")
        list.insert(pos, str)

    elif command == 'd' :
        pos = int( input("  삭제행 번호: ") )
        list.delete(pos)

    elif command == 'r' :
        pos = int( input("  변경행 번호: ") )
        str = input("  변경행 내용: ");
        list.replace(pos, str)

    elif command == 'p' :
        print('Line Editor')
        for line in range (list.size) :
            print('[%2d] '%line, end='')
            print(list.getEntry(line))
        print()

    elif command == 'q' : exit()

    elif command == 'l' :
        # filename = input("  읽어들일 파일 이름: ")
        filename = 'test.txt'
        infile = open(filename , "r")
        lines = infile.readlines();
        for line in lines:
            list.insert(list.size, line.rstrip('₩n'))
        infile.close()

    elif command == 's' :
        # filename = input("  저장할 파일 이름: ")
        filename = 'test.txt'
        outfile = open(filename , "w")
        len = list.size
        for i in range(len) :
            outfile.write(list.getEntry(i)+'₩n')
        outfile.close()
```

<< 새 파일에서 좌측 코드 복사+붙여넣기

파일명: LineEditor.py

실행 결과 예

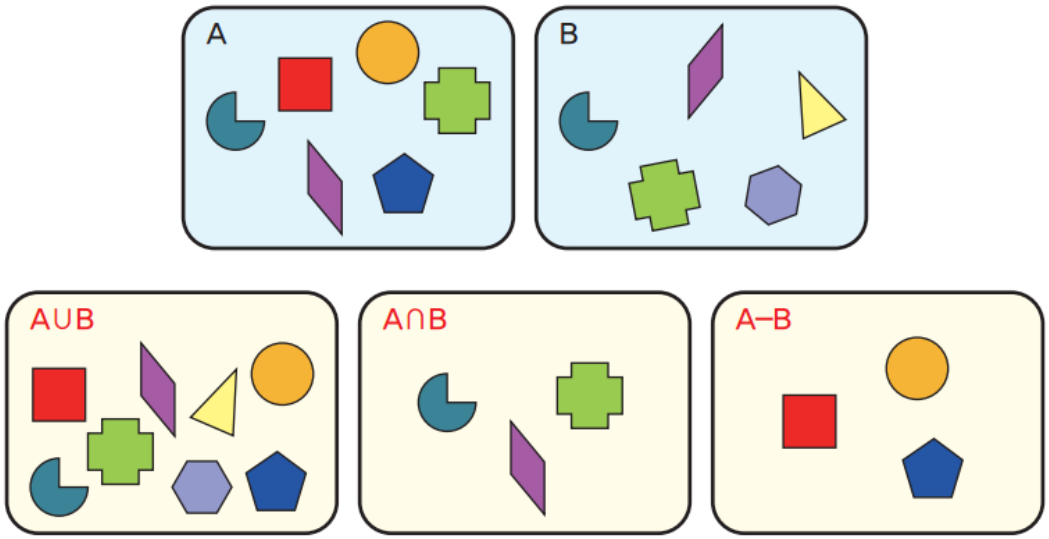
```
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> p
Line Editor

[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> l
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> p
Line Editor
[ 0] data structre
[ 1] welcome
[ 2] python
[ 3] hello
[ 4] world
[ 5] game
[ 6] over
```

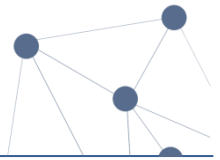
```
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> i
  입력행 번호: 0
  입력행 내용: TEXT
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> r
  변경행 번호: 3
  변경행 내용: text_replace_test
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> d
  삭제행 번호: 5
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, l-파일읽기, s-저장, q-종료=> p
Line Editor
[ 0] TEXT
[ 1] data structre
[ 2] welcome
[ 3] text_replace_test
[ 4] hello
[ 5] game
[ 6] over
```

3.5 집합이란?

- 집합
  - 원소의 중복을 허용하지 않음
  - 원소들 사이에 순서가 없음 → 선형 자료구조가 아님
  - $S = \{item_0, item_1, item_2, \ldots, item_{n-1}\}$



집합의 추상 자료형



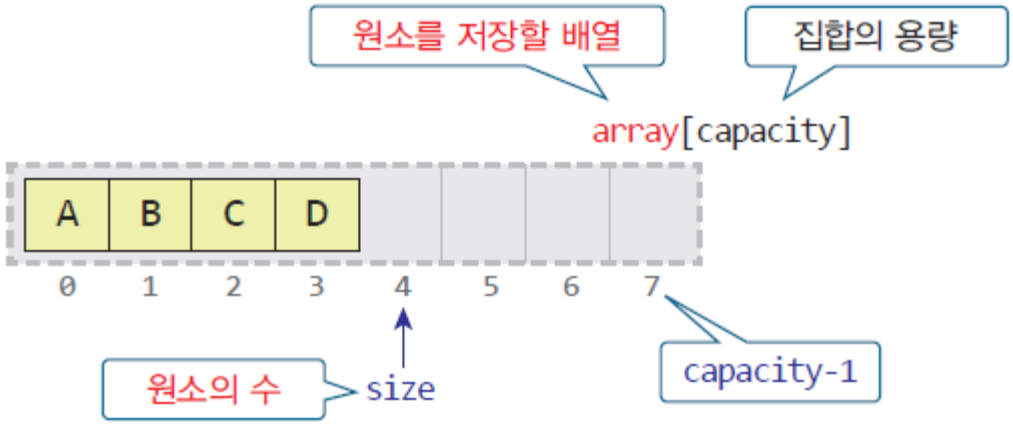
- 데이터
  - 같은 유형의 유일한 요소들의 모임. 원소들은 순서는 없지만 서로 비교할 수는 있어야 함
- 연산
  - contains(e): 집합이 원소 e를 포함하는지를 검사한다.
  - insert(e): 새로운 원소 e를 삽입한다. 중복은 허용하지 않는다.
  - delete(e): 원소 e를 집합에서 꺼내고(삭제) 반환한다.
  - isEmpty(): 공집합인지 검사한다.
  - isFull(): 집합이 가득 차 있는지를 검사한다.
  - union(setB): setB와의 합집합을 만들어 반환한다.
  - intersect(setB): setB와의 교집합을 만들어 반환한다.
  - difference(setB): setB와의 차집합을 만들어 반환한다.

3.6 집합의 구현



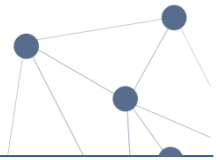
- 집합은 다양한 방법으로 구현할 수 있음
  - 배열, 비트 벡터, 트리, 해싱 구조 등
  - 사용 방법에 따라 연산들의 성능이 달라짐

- 배열을 이용한 집합의 구조



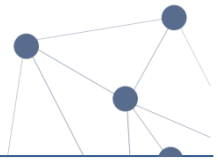
  - 원소들은 정렬하지 않음. 정렬한 경우 → 7.3절

집합의 연산들



- 원소 e가 집합에 있는지 검사하는 contains(e) 연산
  - 순차 탐색, O(n)
- 원소를 삽입하는 insert(e) 연산
  - 중복 검사, O(n)
  - 맨 뒤에 저장
- 원소를 삭제하는 delete(e) 연산
  - 삭제할 원소를 찾기, O(n)
  - 있다면 그 원소 삭제 → 이후의 모든 원소 이동
  - 개선: 맨 뒤의 원소를 삭제할 원소 위치로 옮김
- 기타 연산
  - 합집합 C = A.union(B) → $O(n^2)$
  - 교집합 C = A.intersection(B) → $O(n^2)$
  - 차집합 C = A. difference(B) → $O(n^2)$
- 클래스 코드: 코드 3.6

테스트 프로그램

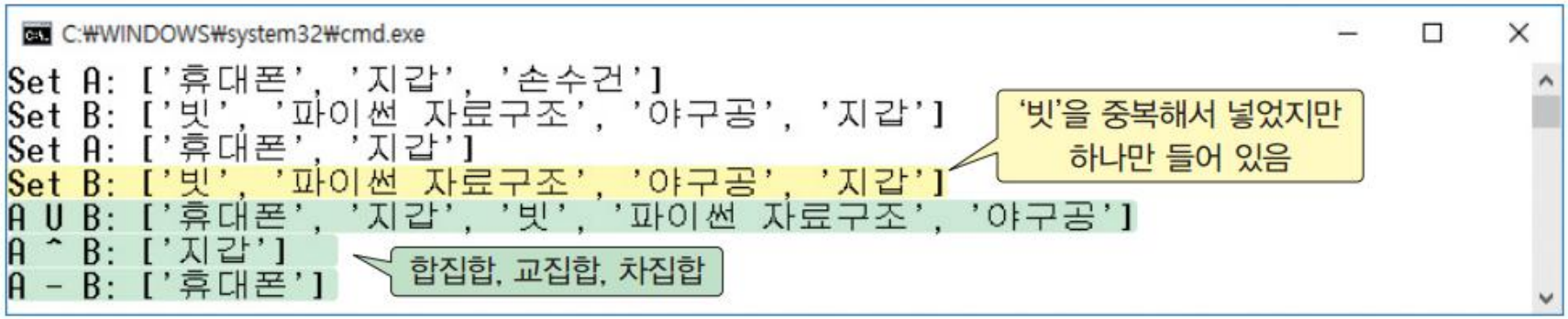


```
setA = Set()
setA.insert('휴대폰')
setA.insert('지갑')
setA.insert('손수건')
setA.display('Set A:')

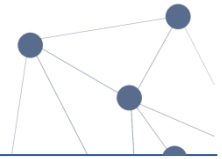
setB = Set()
setB .insert('빗')
setB .insert('파이썬 자료구조')
setB .insert('야구공')
setB .insert('지갑')
setB.display('Set B:')
```

```
setB.insert('빗')
setA.delete('손수건')
setA.delete('발수건')
setA.display('Set A:')
setB.display('Set B:')
```

```
setA.union(setB).display('A U B:')
setA.intersect(setB).display('A ^ B:')
setA.difference(setB).display('A - B:')
```



Summary



파이썬의 리스트: 동적 배열

- 항목 추가 시: new 배열 할당 -> new 에 기존 배열 복사 -> 항목 추가 -> 기존 배열 삭제 -> new 활용

리스트 연산들의 복잡도

- append(e) : O(1) 마지막 위치에 e 추가 (위치 한번에 알 수 있음)
- insert(pos, e) : O(n) pos에 e 추가하고 배열 재정렬
- pop(pos) : O(n) pos에 있는 요소 삭제하고 배열 재정렬

리스트의 배열 구조 구현

- 함수로 구현: 전역변수(데이터) + 연산(함수)
- 클래스로 구현 : 멤버변수(데이터) + 연산(메소드: 멤버 함수)

리스트의 배열 구조 구현

- 라인 편집기 : input() 으로 입력 받은 값에 따라 txt 파일의 리스트 추가/수정/삭제 + 파일 저장 (25~27페이지)



감사합니다!